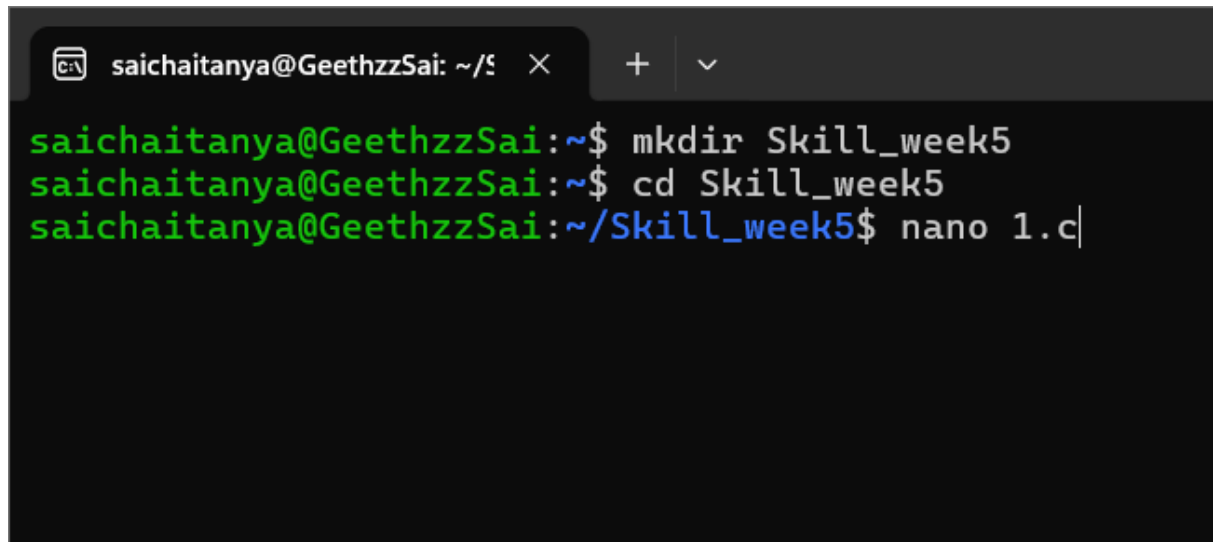


Name: M.Sai Chaitanya ID number: 2520090023

Skill-Week 5

Task 1: Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

1) Create a file by using

A terminal window with a dark background. The title bar shows 'saichaitanya@GeethzzSai: ~/...' with a close button. The terminal contains three lines of commands and their outputs: 1. 'mkdir Skill_week5' is entered, and the prompt changes to '~\$'. 2. 'cd Skill_week5' is entered, and the prompt changes to '~/Skill_week5\$'. 3. 'nano 1.c' is entered, and the prompt remains at '~/Skill_week5\$' with the cursor at the end of the command.

```
saichaitanya@GeethzzSai:~$ mkdir Skill_week5
saichaitanya@GeethzzSai:~$ cd Skill_week5
saichaitanya@GeethzzSai:~/Skill_week5$ nano 1.c|
```

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char command[100];
    char path[PATH_MAX];
    char oldpwd[PATH_MAX];
    char current[PATH_MAX];

    if (getcwd(current, sizeof(current)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(oldpwd, current);

    printf("Mini Shell - Directory Navigation\n");
    printf("Commands: cd <path>, cd -, pwd, exit\n\n");

    while (1)
    {
        if (getcwd(current, sizeof(current)) == NULL)
        {
            perror("getcwd");
            return 1;
        }

        printf("%s$ ", current);
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';
```

```
command[strcspn(command, "\n")] = '\0';

if (strcmp(command, "exit") == 0)
{
    break;
}

if (strcmp(command, "pwd") == 0)
{
    printf("%s\n", current);
    continue;
}

if (strcmp(command, "cd -") == 0)
{
    strcpy(path, oldpwd);
}
else if (strncmp(command, "cd ", 3) == 0)
{
    strcpy(path, command + 3);

    if (strlen(path) == 0)
    {
        printf("cd: missing operand\n");
        continue;
    }
}
else
{
    printf("Invalid command\n");
    continue;
}

if (chdir(path) == -1)
{
    perror("cd");
    continue;
}
```

```

        {
            printf("cd: missing operand\n");
            continue;
        }
    }
    else
    {
        printf("Invalid command\n");
        continue;
    }

    if (chdir(path) == -1)
    {
        perror("cd");
        continue;
    }

    strcpy(oldpwd, current);

    if (getcwd(current, sizeof(current)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    printf("Directory changed to: %s\n", current);
}

return 0;
}

```

After writing the code save the your program by ctrl O and for coming back to command press ctrl X then compile it by the command

```

saichaitanya@GeethzzSai: ~/!$ mkdir Skill_week5
saichaitanya@GeethzzSai: ~/!$ cd Skill_week5
saichaitanya@GeethzzSai: ~/Skill_week5$ nano 1.c
saichaitanya@GeethzzSai: ~/Skill_week5$ gcc 1.c -o 1
saichaitanya@GeethzzSai: ~/Skill_week5$ ./1
Mini Shell - Directory Navigation
Commands: cd <path>, cd -, pwd, exit

/home/saichaitanya/Skill_week5$ |

```

Task 2: Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

1) Create a file

```
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 200

void builtin_pwd()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_cd(char *path)
{
    if (path == NULL)
    {
        printf("cd: missing operand\n");
        return;
    }

    if (chdir(path) == -1)
        perror("cd");
}

void builtin_help()
{
    printf("\nBuilt-in commands:\n");
    printf(" cd <directory>  Change directory\n");
    printf(" pwd              Display current directory\n");
    printf(" help            Display help\n");
    printf(" exit            Exit shell\n\n");
}
```

```
saichaitanya@GeethzzSai:~/Skill_week5$
saichaitanya@GeethzzSai:~/Skill_week5$
saichaitanya@GeethzzSai:~/Skill_week5$ nano 2.c|
```

```
void builtin_exit()
{
    printf("Exiting shell...\n");
    exit(0);
}

int main()
{
    char input[MAX_INPUT];
    char *command;
    char *argument;

    struct builtin
    {
        char *name;
        void (*function)();
    };

    struct builtin dispatch_table[] =
    {
        {"pwd", builtin_pwd},
        {"help", builtin_help},
        {"exit", builtin_exit}
    };

    int table_size = sizeof(dispatch_table) /
                     sizeof(dispatch_table[0]);

    printf("Mini Shell with Built-in Dispatch Table\n");
    printf("Type 'help' to see available commands.\n\n");

    while (1)
    {
        printf("shell$ ");

        if (fgets(input, sizeof(input), stdin) == NULL)
```

```
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

command = strtok(input, " ");
argument = strtok(NULL, " ");

/* Special handling for cd */
if (strcmp(command, "cd") == 0)
{
    builtin_cd(argument);
    continue;
}

/* Search dispatch table */
int found = 0;

for (int i = 0; i < table_size; i++)
{
    if (strcmp(command, dispatch_table[i].name) == 0)
    {
        dispatch_table[i].function();
        found = 1;
        break;
    }
}

if (!found)
{
    printf("Invalid command: %s\n", command);
}

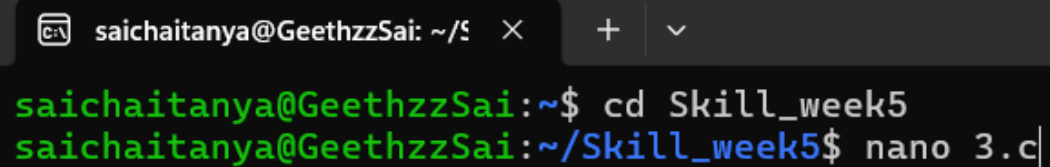
return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/Skill_week5$ nano 2.c
saichaitanya@GeethzzSai:~/Skill_week5$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Skill_week5$ ./2
Mini Shell with Built-in Dispatch Table
Type 'help' to see available commands.

shell$ pwd
/home/saichaitanya/Skill_week5
shell$ hello
Invalid command: hello
shell$ |
```

TASK 3: Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

A terminal window with a dark background. The title bar shows 'saichaitanya@GeethzzSai: ~/5' with a close button and window controls. The terminal content shows the user changing the directory to 'Skill_week5' and then opening a new file '3.c' with the 'nano' editor.

```
saichaitanya@GeethzzSai:~$ cd Skill_week5
saichaitanya@GeethzzSai:~/Skill_week5$ nano 3.c|
```

CODE:


```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main()
{
    char cwd[PATH_MAX];
    char command[100];

    printf("Simple Directory Shell\n");
    printf("Commands: pwd, exit\n\n");

    while (1)
    {
        /* Retrieve current directory */
        if (getcwd(cwd, sizeof(cwd)) == NULL)
        {
            perror("getcwd");
            return 1;
        }

        /* Display current path */
        printf("%s$ ", cwd);

        if (fgets(command, sizeof(command), stdin) == NULL)
        {
            printf("\nInput ended.\n");
            break;
        }

        command[strcspn(command, "\n")] = '\0';

        /* Exit request */
        if (strcmp(command, "exit") == 0)
        {

```

```

        command[strcspn(command, "\n")] = '\0';

        /* Exit request */
        if (strcmp(command, "exit") == 0)
        {
            printf("Exit request received.\n");
            printf("Cleaning up resources...\n");
            printf("Shell state saved.\n");
            printf("Exiting successfully.\n");
            break;
        }

        /* Display current directory */
        else if (strcmp(command, "pwd") == 0)
        {
            printf("Current directory: %s\n", cwd);
        }

        else
        {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
|

```

OUTPUT:

```

saichaitanya@GeethzzSai: ~/5  ×  +  v
saichaitanya@GeethzzSai:~$ cd Skill_week5
saichaitanya@GeethzzSai:~/Skill_week5$ nano 3.c
saichaitanya@GeethzzSai:~/Skill_week5$ gcc 3.c -o 3
saichaitanya@GeethzzSai:~/Skill_week5$ ./3
Simple Directory Shell
Commands: pwd, exit

/home/saichaitanya/Skill_week5$ pwd
Current directory: /home/saichaitanya/Skill_week5
/home/saichaitanya/Skill_week5$ |

```

```
saichaitanya@GeethzzSai: ~/!  ×  +  ∨  
saichaitanya@GeethzzSai:~$ cd Skill_week5  
saichaitanya@GeethzzSai:~/Skill_week5$ nano 3.c  
saichaitanya@GeethzzSai:~/Skill_week5$ gcc 3.c -o 3  
saichaitanya@GeethzzSai:~/Skill_week5$ ./3  
Simple Directory Shell  
Commands: pwd, exit  
  
/home/saichaitanya/Skill_week5$ pwd  
Current directory: /home/saichaitanya/Skill_week5  
/home/saichaitanya/Skill_week5$ exit  
Exit request received.  
Cleaning up resources...  
Shell state saved.  
Exiting successfully.  
saichaitanya@GeethzzSai:~/Skill_week5$ |
```

TASK 4: Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
saichaitanya@GeethzzSai:~/Skill_week5$  
saichaitanya@GeethzzSai:~/Skill_week5$  
saichaitanya@GeethzzSai:~/Skill_week5$ nano 4.c|
```

CODE:

```
GNU nano 8.7.1 4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int valid_name(const char *name)
{
    int i;

    if (name[0] == '\\0')
        return 0;

    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    for (i = 1; name[i] != '\\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main()
{
    char input[200];

    printf("Environment Variable Shell\\n");
    printf("Commands: export NAME=VALUE, print NAME, test, exit\\n\\n");

    while (1)
    {
        printf("shell$ ");
```

```
printf("shell$ ");

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

/* Exit */
if (strcmp(input, "exit") == 0)
{
    printf("Exiting shell...\n");
    break;
}

/* Export command */
if (strncmp(input, "export ", 7) == 0)
{
    char *assignment = input + 7;
    char *equal = strchr(assignment, '=');

    if (equal == NULL)
    {
        printf("Invalid export syntax.\n");
        continue;
    }

    *equal = '\0';

    char *name = assignment;
    char *value = equal + 1;

    if (!valid_name(name))
    {
        printf("Invalid variable name.\n");
        continue;
    }
}
```

```
        continue;
    }

    if (setenv(name, value, 1) == -1)
    {
        perror("setenv");
        continue;
    }

    printf("Exported: %s=%s\n", name, value);
}

/* Print environment variable */
else if (strncmp(input, "print ", 6) == 0)
{
    char *name = input + 6;
    char *value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable '%s' is not set.\n", name);
}

/* Test variable in child process */
else if (strcmp(input, "test") == 0)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
```

```

        if (pid == 0)
        {
            char *value = getenv("MYVAR");

            if (value != NULL)
                printf("Child process received MYVAR=%s\n", value);
            else
                printf("Child process: MYVAR not found.\n");

            exit(0);
        }
        else
        {
            wait(NULL);
        }
    }

    else
    {
        printf("Invalid command.\n");
    }
}

return 0;
}

```

OUTPUT:

```

saichaitanya@GeethzzSai:~/Skill_week5$ ./4
saichaitanya@GeethzzSai:~/Skill_week5$ nano 4.c
saichaitanya@GeethzzSai:~/Skill_week5$ gcc 4.c -o 4
saichaitanya@GeethzzSai:~/Skill_week5$ ./4
Environment Variable Shell
Commands: export NAME=VALUE, print NAME, test, exit

shell$ pwd
Invalid command.
shell$ hello
Invalid command.
shell$

```